The Developer's Craft

Phase 2: Patterns, Pitfalls & Professional Practices

Mastering the Art of Building Software That Works

Beyond Syntax: The Thinking Process

Real Projects, Real Problems, Real Solutions

March 5, 2026

Contents

The Gap Between Tutorial & Reality

Tutorial Code:

- Perfect input assume karta hai
- Memory hamesha available hai
- Files hamesha exist karti hain
- Network kabhi fail nahi hota
- User hamesha sahi button dabata hai

Real World:

- User "abc" bhi age mein daal sakta hai
- Disk full ho sakti hai
- File delete ho gayi kisi aur ne
- WiFi 2 second ke liye drop hua
- User ne 1000 files ek saath upload kar di

Is Guide ka Goal: Aapko woh **mental models** aur **patterns** sikhana jo senior developers use karte hain jab sab kuchh break ho raha ho.

Part I

The Art of Expecting Failure

Chapter 1

When Everything Goes Wrong: Defensive Programming

1.1 War Story 1: The 3 AM Phone Call

Real War Story: Production Disaster

Scenario: E-commerce company, Diwali sale, 10,000 orders/minute.
The Bug:

```
// Junior developer ka code
void processOrder(Order order) {
    Database.save(order);            // Success assume kiya
    PaymentGateway.charge(order);    // Yahan fail hua
    EmailService.sendConfirmation(order);  // Bhej diya galat
        confirmation!
    Inventory.reduce(order.items);  // Stock kam ho gaya,
        payment nahi aaya
}
```

Result:

- 5000 orders database mein, payment nahi mila
- 5000 customers ko "Order Confirmed" email aa gaya
- Inventory kam ho gaya, paise nahi aaye
- Company ko 20 lakh ka loss + reputation damage

Root Cause: Developer ne assume kiya ki har step hamesha succeed hoga.

1.2 The Pattern: Transactional Safety

The Pattern: How Pros Solve This

The Golden Rule: Har operation do categories mein:

1. **Safe Operations:** Kuchh bhi ho, data corrupt nahi hoga (read-only)
2. **Dangerous Operations:** Fail ho sakti hain, rollback chahiye (write operations)

The Strategy:

1. Validate everything first (cheap checks)
2. Prepare all data (no external calls yet)
3. Execute critical section (atomic operation)
4. If any step fails, undo everything (compensating transactions)

```cpp
// production_order_system.hpp
#ifndef ORDER_SYSTEM_HPP
#define ORDER_SYSTEM_HPP

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <functional>
#include <exception>
#include <sstream>

using namespace std;

// ==================== EXCEPTION HIERARCHY ====================

class BusinessException : public exception {
protected:
    string message;
    string errorCode;
    string context;

public:
    BusinessException(const string& msg, const string& code, const string& ctx = "")
        : message(msg), errorCode(code), context(ctx) {}

    const char* what() const noexcept override { return message.c_str(); }
    string getCode() const { return errorCode; }
    string getContext() const { return context; }
```

```

    string toString() const {
        ostringstream oss;
        oss << "[" << errorCode << "] " << message;
        if(!context.empty()) oss << " | Context: " << context;
        return oss.str();
    }
};

class ValidationException : public BusinessException {
public:
    ValidationException(const string& field, const string& reason)
        : BusinessException("Validation failed: " + field + " - " +
            reason,
                            "VAL_001", field) {}
};

class PaymentException : public BusinessException {
public:
    PaymentException(const string& reason, const string&
       transactionId)
        : BusinessException("Payment failed: " + reason, "PAY_001",
                            "TxID: " + transactionId) {}
};

class InventoryException : public BusinessException {
public:
    InventoryException(const string& productId, int requested, int
       available)
        : BusinessException("Insufficient inventory", "INV_001",
                            "Product: " + productId +
                            ", Requested: " + to_string(requested) +
                            ", Available: " + to_string(available))
                               {}
};

// ==================== RESULT PATTERN ====================

template<typename T>
class Result {
private:
    bool success;
    T value;
    string error;
    string errorCode;

    Result(bool s, const T& v, const string& e, const string& c)
        : success(s), value(v), error(e), errorCode(c) {}

public:
    static Result<T> Ok(const T& val) {
```

```
        return Result(true, val, "", "");
    }

    static Result<T> Fail(const string& err, const string& code) {
        return Result(false, T(), err, code);
    }

    bool isSuccess() const { return success; }
    bool isFailure() const { return !success; }

    T getValue() const {
        if(!success) throw runtime_error("Called getValue on failed Result");
        return value;
    }

    string getError() const { return error; }
    string getErrorCode() const { return errorCode; }

    // Functional style chaining
    template<typename Func>
    auto map(Func f) -> Result<decltype(f(value))> {
        if(!success) return Result<decltype(f(value))>::Fail(error, errorCode);
        try {
            return Result<decltype(f(value))>::Ok(f(value));
        } catch(const exception& e) {
            return Result<decltype(f(value))>::Fail(e.what(), "MAP_ERROR");
        }
    }

    template<typename Func>
    Result<T> orElse(Func f) {
        if(!success) return f();
        return *this;
    }
};

// Void specialization
template<>
class Result<void> {
private:
    bool success;
    string error;
    string errorCode;

public:
    Result(bool s, const string& e, const string& c)
        : success(s), error(e), errorCode(c) {}

```

```
    static Result<void> Ok() { return Result(true, "", ""); }
    static Result<void> Fail(const string& e, const string& c) {
        return Result(false, e, c);
    }

    bool isSuccess() const { return success; }
    bool isFailure() const { return !success; }
    string getError() const { return error; }
};

#endif
```

```
// order_system.cpp - Production Grade Implementation
#include "production_order_system.hpp"
#include <map>
#include <chrono>

// Mock external services
class PaymentGateway {
public:
    static Result<string> charge(double amount, const string&
       cardToken) {
        // Simulate 10% failure rate
        if(rand() % 10 == 0) {
            return Result<string>::Fail("Card declined", "
               PAY_DECLINED");
        }
        if(amount > 100000) {
            return Result<string>::Fail("Amount exceeds limit", "
               PAY_LIMIT");
        }
        return Result<string>::Ok("TXN_" + to_string(rand()));
    }

    static Result<void> refund(const string& transactionId) {
        cout << "  [Payment] Refunding " << transactionId << endl;
        return Result<void>::Ok();
    }
};

class InventoryService {
    static map<string, int> stock;

public:
    static Result<int> checkAvailability(const string& productId) {
        auto it = stock.find(productId);
        if(it == stock.end()) {
            return Result<int>::Fail("Product not found", "
               INV_NOT_FOUND");
        }
        return Result<int>::Ok(it->second);
```

```
    }

    static Result<void> reserve(const string& productId, int qty) {
        auto it = stock.find(productId);
        if(it == stock.end() || it->second < qty) {
            return Result<void>::Fail("Insufficient stock", "INV_SHORTAGE");
        }
        it->second -= qty;
        cout << "  [Inventory] Reserved " << qty << "x " << productId
             << " (Remaining: " << it->second << ")" << endl;
        return Result<void>::Ok();
    }

    static Result<void> release(const string& productId, int qty) {
        stock[productId] += qty;
        cout << "  [Inventory] Released " << qty << "x " << productId << endl;
        return Result<void>::Ok();
    }
};

map<string, int> InventoryService::stock = {
    {"PROD_001", 100},
    {"PROD_002", 50},
    {"PROD_003", 0}  // Out of stock
};

class EmailService {
public:
    static void sendOrderConfirmation(const string& email, const string& orderId) {
        cout << "  [Email] Confirmation sent to " << email << " for order " << orderId << endl;
    }

    static void sendFailureNotification(const string& email, const string& reason) {
        cout << "  [Email] Failure notification: " << reason << endl;
    }
};

// ==================== ORDER PROCESSOR ====================

struct OrderItem {
    string productId;
    int quantity;
    double price;
};
```

```

struct Order {
    string orderId;
    string customerEmail;
    string cardToken;
    vector<OrderItem> items;
    double totalAmount;
};

class OrderProcessor {
public:
    // The CORRECT way: Validate -> Prepare -> Execute with rollback
    Result<string> processOrder(const Order& order) {
        cout << "\n=== Processing Order " << order.orderId << " ===" << endl;

        // STEP 1: Validation (Cheap, no side effects)
        auto validation = validateOrder(order);
        if(validation.isFailure()) {
            logFailure(order, "VALIDATION", validation.getError());
            return Result<string>::Fail(validation.getError(), validation.getErrorCode());
        }

        // STEP 2: Check inventory (Still no side effects)
        vector<string> reservedProducts;
        for(const auto& item : order.items) {
            auto avail = InventoryService::checkAvailability(item.productId);
            if(avail.isFailure() || avail.getValue() < item.quantity) {
                return Result<string>::Fail(
                    "Product unavailable: " + item.productId, "INV_CHECK_FAIL");
            }
        }

        // STEP 3: Reserve inventory (First side effect - needs rollback)
        for(const auto& item : order.items) {
            auto reserve = InventoryService::reserve(item.productId, item.quantity);
            if(reserve.isSuccess()) {
                reservedProducts.push_back(item.productId);
            } else {
                // CRITICAL: Rollback already reserved items
                rollbackInventory(order, reservedProducts);
                return Result<string>::Fail("Reservation failed", "RESERVE_FAIL");
            }
```

```
        }

        // STEP 4: Process payment (Most critical - money involved)
        auto payment = PaymentGateway::charge(order.totalAmount,
            order.cardToken);
        if(payment.isFailure()) {
            // CRITICAL: Release inventory since payment failed
            rollbackInventory(order, reservedProducts);
            EmailService::sendFailureNotification(order.
                customerEmail,
                "Payment failed: " + payment.getError());
            return Result<string>::Fail(payment.getError(), payment
                .getErrorCode());
        }

        string transactionId = payment.getValue();

        // STEP 5: Save to database (Final persistence)
        auto saveResult = saveOrderToDatabase(order, transactionId)
            ;
        if(saveResult.isFailure()) {
            // CRITICAL: Refund payment + release inventory
            PaymentGateway::refund(transactionId);
            rollbackInventory(order, reservedProducts);
            return Result<string>::Fail("Database error", "DB_FAIL"
                );
        }

        // STEP 6: Send confirmation (Non-critical, can fail
            silently)
        try {
            EmailService::sendOrderConfirmation(order.customerEmail
                , order.orderId);
        } catch(...) {
            // Log but don't fail the order
            cout << "  [Warning] Email failed but order succeeded"
                << endl;
        }

        cout << "=== Order " << order.orderId << " Completed
            Successfully ===" << endl;
        return Result<string>::Ok(transactionId);
    }

private:
    Result<void> validateOrder(const Order& order) {
        if(order.orderId.empty()) {
            return Result<void>::Fail("Order ID missing", "
                VAL_NO_ID");
        }
        if(order.customerEmail.find('@') == string::npos) {
```

```
            return Result<void>::Fail("Invalid email", "VAL_EMAIL");
        }
        if(order.totalAmount <= 0) {
            return Result<void>::Fail("Invalid amount", "VAL_AMOUNT");
        }
        if(order.items.empty()) {
            return Result<void>::Fail("No items in order", "VAL_EMPTY");
        }
        return Result<void>::Ok();
    }

    void rollbackInventory(const Order& order, const vector<string>& reserved) {
        cout << "  [Rollback] Releasing inventory..." << endl;
        for(size_t i = 0; i < reserved.size(); ++i) {
            InventoryService::release(order.items[i].productId, order.items[i].quantity);
        }
    }

    Result<void> saveOrderToDatabase(const Order& order, const string& txnId) {
        // Simulate DB operation
        if(rand() % 20 == 0) {  // 5% DB failure
            return Result<void>::Fail("Connection timeout", "DB_TIMEOUT");
        }
        cout << "  [Database] Order saved with transaction " << txnId << endl;
        return Result<void>::Ok();
    }

    void logFailure(const Order& order, const string& stage, const string& reason) {
        cerr << "  [ERROR] Order " << order.orderId
             << " failed at " << stage << ": " << reason << endl;
    }
};

// ==================== TESTING ====================

int main() {
    srand(time(nullptr));
    OrderProcessor processor;

    // Test Case 1: Valid order
    Order order1;
    order1.orderId = "ORD_001";
```

```
    order1.customerEmail = "customer@example.com";
    order1.cardToken = "tok_visa";
    order1.items = {{"PROD_001", 2, 1000.0}, {"PROD_002", 1, 500.0}};
    order1.totalAmount = 2500.0;

    auto result1 = processor.processOrder(order1);
    cout << "Result: " << (result1.isSuccess() ? "SUCCESS" : "FAILED")
         << " - " << (result1.isSuccess() ? result1.getValue() : result1.getError()) << endl;

    // Test Case 2: Out of stock
    Order order2;
    order2.orderId = "ORD_002";
    order2.customerEmail = "test@test.com";
    order2.cardToken = "tok_master";
    order2.items = {{"PROD_003", 1, 100.0}};  // PROD_003 has 0 stock
    order2.totalAmount = 100.0;

    auto result2 = processor.processOrder(order2);
    cout << "\nResult: " << (result2.isSuccess() ? "SUCCESS" : "FAILED")
         << " - " << result2.getError() << endl;

    // Test Case 3: Run multiple to see failure handling
    cout << "\n=== Running 5 orders to see failure scenarios ===" << endl;
    for(int i = 3; i <= 7; ++i) {
        Order o;
        o.orderId = "ORD_00" + to_string(i);
        o.customerEmail = "user" + to_string(i) + "@test.com";
        o.cardToken = "tok_rand";
        o.items = {{"PROD_001", 1, 500.0}};
        o.totalAmount = 500.0;

        auto r = processor.processOrder(o);
        cout << "Order " << o.orderId << ": "
             << (r.isSuccess() ? "   " : "   ") << endl;
    }

    return 0;
}
```

Pro Tip: The 10x Developer Secret

The Result Pattern Benefits:

1. **No exceptions for control flow:** Exceptions expensive hain, Result cheap

2. **Forced error handling:** Compiler forces you to check isSuccess()
3. **Chain operations:** map(), orElse() se functional style
4. **Clear error context:** Har error ke saath code aur context

JavaScript mein Promise, Rust mein Result, C++ mein yeh pattern - sab same concept!

Chapter 2

The Debugging Mindset: When You Have No Idea What's Wrong

2.1 War Story 2: The Heisenbug

Real War Story: Production Disaster

Scenario: Banking software, random crashes in production.
Symptoms:

- Debug build mein kabhi crash nahi hota
- Release build mein 1 in 1000 transactions pe crash
- Stack trace garbage dikhata hai
- Logs mein kuchh suspicious nahi
- Developer ke machine pe reproduce nahi hota

3 weeks waste karne ke baad mila:

```
// The culprit
void processTransaction(Transaction* tx) {
    // Thread A
    if(tx->isValid()) {            // Check 1
        // Thread B ne yahan beech mein tx invalid kar diya!
        process(tx->amount);       // Crash - tx null ho gaya!
    }
}
```

The Bug: Race condition between threads. Check karne ke baad, use karne se pehle, dusra thread data delete kar raha tha.
Lesson: Debugging mein sabse bada dushman "assumption" hai.

Debug Mindset: When Everything Breaks

The Scientific Method of Debugging:

1. **Observe:** Exactly kya ho raha hai? "Crash" nahi, "Segmentation fault at address 0x00000000 in processTransaction line 45"
2. **Hypothesize:** 3 possible reasons:
   - Null pointer dereference
   - Use-after-free
   - Stack corruption
3. **Experiment:** Har hypothesis test karo:
   - Null check add karo → Still crash? → Hypothesis wrong
   - Valgrind chalao → Invalid read? → Use-after-free confirm
4. **Verify:** Fix karke 1000 baar run karo, no crash = solved

Golden Rule: Random changes mat karo. Har change ek hypothesis test hona chahiye.

```cpp
// debugging_tools.hpp - Professional Debugging Toolkit
#ifndef DEBUGGING_TOOLS_HPP
#define DEBUGGING_TOOLS_HPP

#include <iostream>
#include <string>
#include <chrono>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <source_location>
#include <stacktrace>

using namespace std;

// ==================== ASSERTION WITH CONTEXT ====================

#define ASSERT(condition, message) \
    do { \
        if(!(condition)) { \
            cerr << "\n========== ASSERTION FAILED ==========" << endl; \
            cerr << "File: " << __FILE__ << endl; \
            cerr << "Line: " << __LINE__ << endl; \
            cerr << "Function: " << __func__ << endl; \
            cerr << "Condition: " << #condition << endl; \
            cerr << "Message: " << message << endl; \
            cerr << "Stack trace:" << endl; \
```

```
            cerr << stacktrace::current() << endl; \
            cerr << "=====================================" << endl; \
            abort(); \
        } \
    } while(0)

// ==================== SCOPE LOGGER (RAII) ====================

class ScopeLogger {
private:
    string name;
    chrono::steady_clock::time_point start;
    bool completed;

public:
    explicit ScopeLogger(const string& n) : name(n), completed(false) {
        start = chrono::steady_clock::now();
        log("ENTER", "");
    }

    ~ScopeLogger() {
        if(!completed) {
            log("EXIT", " (exception?)");
        }
    }

    void success() {
        completed = true;
        auto duration = chrono::steady_clock::now() - start;
        auto ms = chrono::duration_cast<chrono::milliseconds>(duration).count();
        log("SUCCESS", " [" + to_string(ms) + "ms]");
    }

    void fail(const string& reason) {
        completed = true;
        log("FAILED", ": " + reason);
    }

private:
    void log(const string& event, const string& extra) {
        auto now = chrono::system_clock::now();
        auto time = chrono::system_clock::to_time_t(now);

        cerr << "[" << put_time(localtime(&time), "%H:%M:%S") << "] "
             << event << " " << name << extra << endl;
    }
};
```

```

#define SCOPE_LOG(name) ScopeLogger _scopeLog(name)
#define SCOPE_SUCCESS() _scopeLog.success()
#define SCOPE_FAIL(msg) _scopeLog.fail(msg)

// ==================== STATE INSPECTOR ====================

template<typename T>
class StateInspector {
private:
    string name;
    T* ptr;
    vector<pair<string, string>> history;

public:
    StateInspector(const string& n, T* p) : name(n), ptr(p) {
        record("CONSTRUCTED");
    }

    ~StateInspector() {
        record("DESTRUCTED");
        printHistory();
    }

    void record(const string& event) {
        auto now = chrono::system_clock::now();
        auto time = chrono::system_clock::to_time_t(now);

        ostringstream oss;
        oss << put_time(localtime(&time), "%H:%M:%S");

        string value = ptr ? to_string(*ptr) : "NULL";
        history.push_back({oss.str(), event + " | Value: " + value
            });
    }

    void printHistory() {
        cerr << "\n=== State History: " << name << " ===" << endl;
        for(const auto& [time, event] : history) {
            cerr << time << " | " << event << endl;
        }
        cerr << "============================" << endl;
    }
};

// ==================== RACE CONDITION DETECTOR
    ====================

#include <mutex>
#include <thread>
#include <unordered_map>
```

```

class ThreadSafetyChecker {
private:
    static unordered_map<void*, thread::id> ownership;
    static mutex mapMutex;

public:
    static void checkAccess(void* ptr, const string& operation) {
        lock_guard<mutex> lock(mapMutex);
        auto it = ownership.find(ptr);
        auto currentId = this_thread::get_id();

        if(it == ownership.end()) {
            ownership[ptr] = currentId;
        } else if(it->second != currentId) {
            cerr << "\  n      POTENTIAL RACE CONDITION!" << endl;
            cerr << "Pointer: " << ptr << endl;
            cerr << "Operation: " << operation << endl;
            cerr << "Owned by thread: " << it->second << endl;
            cerr << "Accessed by thread: " << currentId << endl;
            cerr << "Stack: " << stacktrace::current() << endl;
        }
    }

    static void release(void* ptr) {
        lock_guard<mutex> lock(mapMutex);
        ownership.erase(ptr);
    }
};

unordered_map<void*, thread::id> ThreadSafetyChecker::ownership;
mutex ThreadSafetyChecker::mapMutex;

#define CHECK_THREAD_SAFETY(ptr, op) ThreadSafetyChecker::
   checkAccess(ptr, op)

#endif
```

```
// debugging_demo.cpp - Using the tools
#include "debugging_tools.hpp"
#include <thread>
#include <vector>

// Example 1: Finding memory corruption
void demonstrateMemoryBug() {
    cout << "\n=== Demo 1: Memory Bug Detection ===" << endl;

    int* data = new int(42);
    StateInspector<int> inspector("data", data);

    // Normal operations
```

```
    *data = 100;
    inspector.record("Modified to 100");

    // Simulate bug: double delete
    delete data;
    inspector.record("Deleted");

    // Oops! Use after free
    // *data = 200;  // This would crash or corrupt
}

// Example 2: Scope tracking
Result<double> divideSafely(double a, double b) {
    SCOPE_LOG("divideSafely");

    if(b == 0) {
        SCOPE_FAIL("Division by zero");
        return Result<double>::Fail("Cannot divide by zero", "
           MATH_001");
    }

    double result = a / b;
    SCOPE_SUCCESS();
    return Result<double>::Ok(result);
}

// Example 3: Race condition detection
class UnsafeCounter {
private:
    int count = 0;

public:
    void increment() {
        CHECK_THREAD_SAFETY(this, "increment");
        // Missing mutex! This is unsafe
        int temp = count;
        this_thread::sleep_for(chrono::milliseconds(1));
        count = temp + 1;
    }

    int get() const { return count; }
};

void demonstrateRaceCondition() {
    cout << "\n=== Demo 3: Race Condition ===" << endl;

    UnsafeCounter counter;
    vector<thread> threads;

    for(int i = 0; i < 10; ++i) {
        threads.emplace_back([&counter]() {
```

```
            for(int j = 0; j < 100; ++j) {
                counter.increment();
            }
        });
    }

    for(auto& t : threads) t.join();

    cout << "Expected: 1000, Actual: " << counter.get() << endl;
    cout << "Difference shows race condition!" << endl;
}

int main() {
    cout << "     Debugging Tools Demo" << endl;

    // Test 1
    demonstrateMemoryBug();

    // Test 2
    cout << "\n=== Demo 2: Scope Tracking ===" << endl;
    auto r1 = divideSafely(10, 2);
    auto r2 = divideSafely(10, 0);

    // Test 3
    demonstrateRaceCondition();

    // Test 4: Assertion
    cout << "\n=== Demo 4: Assertions ===" << endl;
    int* ptr = nullptr;
    // ASSERT(ptr != nullptr, "Pointer should not be null");  // Would abort

    cout << "\ n   All demos completed!" << endl;
    return 0;
}
```

Chapter 3

Design Patterns: Real Usage, Not Theory

3.1 The Patterns That Actually Matter

The Pattern: How Pros Solve This

80% of problems solve ho jaate hain 4 patterns se:

1. **Factory:** Object creation complex hai, centralize karo
2. **Strategy:** Algorithm interchangeable hona chahiye
3. **Observer:** Event-driven architecture (notifications)
4. **Decorator:** Functionality add karo bina modify kiye

Baki 16 GoF patterns rarely use hote hain daily coding mein.

```cpp
// real_patterns.hpp - Patterns in Action
#ifndef REAL_PATTERNS_HPP
#define REAL_PATTERNS_HPP

#include <iostream>
#include <vector>
#include <memory>
#include <functional>
#include <map>
#include <string>

using namespace std;

// ==================== FACTORY PATTERN ====================
// Use case: Different payment methods, different database types

class DatabaseConnection {
public:
    virtual void connect() = 0;
```

```
    virtual void query(const string& sql) = 0;
    virtual ~DatabaseConnection() = default;
};

class SQLiteConnection : public DatabaseConnection {
public:
    void connect() override {
        cout << "Connecting to SQLite..." << endl;
    }
    void query(const string& sql) override {
        cout << "SQLite executing: " << sql << endl;
    }
};

class PostgreSQLConnection : public DatabaseConnection {
public:
    void connect() override {
        cout << "Connecting to PostgreSQL..." << endl;
    }
    void query(const string& sql) override {
        cout << "PostgreSQL executing: " << sql << endl;
    }
};

class DatabaseFactory {
public:
    static unique_ptr<DatabaseConnection> create(const string& type) {
        if(type == "sqlite") return make_unique<SQLiteConnection>();
        if(type == "postgres") return make_unique<PostgreSQLConnection>();
        throw runtime_error("Unknown database type: " + type);
    }
};

// ==================== STRATEGY PATTERN ====================
// Use case: Different pricing algorithms, different sorting

class PricingStrategy {
public:
    virtual double calculatePrice(double basePrice, int quantity) = 0;
    virtual ~PricingStrategy() = default;
};

class RegularPricing : public PricingStrategy {
public:
    double calculatePrice(double basePrice, int quantity) override {
        return basePrice * quantity;
```

```
    }
};

class BulkDiscountPricing : public PricingStrategy {
    double threshold;
    double discountPercent;

public:
    BulkDiscountPricing(double t, double d) : threshold(t),
       discountPercent(d) {}

    double calculatePrice(double basePrice, int quantity) override
       {
        double total = basePrice * quantity;
        if(quantity >= threshold) {
            total *= (1 - discountPercent);
        }
        return total;
    }
};

class SeasonalPricing : public PricingStrategy {
    double seasonalMultiplier;

public:
    explicit SeasonalPricing(double m) : seasonalMultiplier(m) {}

    double calculatePrice(double basePrice, int quantity) override
       {
        return basePrice * quantity * seasonalMultiplier;
    }
};

// Context
class ShoppingCart {
    unique_ptr<PricingStrategy> strategy;
    vector<pair<string, double>> items;  // name, price

public:
    void setStrategy(unique_ptr<PricingStrategy> s) {
        strategy = move(s);
    }

    void addItem(const string& name, double price) {
        items.push_back({name, price});
    }

    double checkout() {
        if(!strategy) throw runtime_error("No pricing strategy set"
           );

```

```
        double total = 0;
        for(const auto& [name, price] : items) {
            total += strategy->calculatePrice(price, 1);
        }
        return total;
    }
};

// ==================== OBSERVER PATTERN ====================
// Use case: Event systems, notifications, MVC updates

class Observer {
public:
    virtual void onEvent(const string& eventType, const map<string,
        string>& data) = 0;
    virtual ~Observer() = default;
};

class Subject {
private:
    vector<Observer*> observers;
    map<string, vector<Observer*>> specificObservers;

public:
    void attach(Observer* o) {
        observers.push_back(o);
    }

    void attach(const string& eventType, Observer* o) {
        specificObservers[eventType].push_back(o);
    }

    void detach(Observer* o) {
        observers.erase(remove(observers.begin(), observers.end(),
           o), observers.end());
    }

    void notify(const string& eventType, const map<string, string>&
        data) {
        // Notify specific observers
        auto it = specificObservers.find(eventType);
        if(it != specificObservers.end()) {
            for(auto* o : it->second) {
                o->onEvent(eventType, data);
            }
        }

        // Notify general observers
        for(auto* o : observers) {
            o->onEvent(eventType, data);
        }
```

```
    }
};

// Concrete observers
class EmailNotifier : public Observer {
public:
    void onEvent(const string& type, const map<string, string>&
        data) override {
        if(type == "ORDER_PLACED") {
            cout << "     Email: Order confirmation to " << data.at
                ("email") << endl;
        }
    }
};

class InventoryUpdater : public Observer {
public:
    void onEvent(const string& type, const map<string, string>&
        data) override {
        if(type == "ORDER_PLACED") {
            cout << "     Inventory: Reducing stock for " << data.
                at("product") << endl;
        }
    }
};

class AnalyticsLogger : public Observer {
public:
    void onEvent(const string& type, const map<string, string>&
        data) override {
        cout << "     Analytics: Event " << type << " logged" <<
            endl;
    }
};

// ==================== DECORATOR PATTERN ====================
// Use case: Adding features without inheritance hell

class DataSource {
public:
    virtual void write(const string& data) = 0;
    virtual string read() = 0;
    virtual ~DataSource() = default;
};

class FileDataSource : public DataSource {
    string filename;

public:
    explicit FileDataSource(const string& f) : filename(f) {}

```

```
    void write(const string& data) override {
        cout << "Writing to file " << filename << endl;
    }

    string read() override {
        return "Data from " + filename;
    }
};

// Decorator base
class DataSourceDecorator : public DataSource {
protected:
    unique_ptr<DataSource> wrappee;

public:
    explicit DataSourceDecorator(unique_ptr<DataSource> w) :
       wrappee(move(w)) {}
};

// Concrete decorators
class EncryptionDecorator : public DataSourceDecorator {
public:
    using DataSourceDecorator::DataSourceDecorator;

    void write(const string& data) override {
        cout << "     Encrypting data..." << endl;
        wrappee->write("encrypted(" + data + ")");
    }

    string read() override {
        string data = wrappee->read();
        cout << "     Decrypting data..." << endl;
        return data;
    }
};

class CompressionDecorator : public DataSourceDecorator {
public:
    using DataSourceDecorator::DataSourceDecorator;

    void write(const string& data) override {
        cout << "         Compressing data..." << endl;
        wrappee->write("compressed(" + data + ")");
    }

    string read() override {
        string data = wrappee->read();
        cout << "         Decompressing data..." << endl;
        return data;
    }
};
```

```

#endif
```

```
// patterns_demo.cpp
#include "real_patterns.hpp"

int main() {
    cout << "=== Factory Pattern ===" << endl;
    auto db = DatabaseFactory::create("postgres");
    db->connect();
    db->query("SELECT * FROM users");

    cout << "\n=== Strategy Pattern ===" << endl;
    ShoppingCart cart;
    cart.addItem("Laptop", 1000);
    cart.addItem("Mouse", 50);

    // Regular pricing
    cart.setStrategy(make_unique<RegularPricing>());
    cout << "Regular total: $" << cart.checkout() << endl;

    // Bulk discount (buy 2+ get 10% off)
    cart.setStrategy(make_unique<BulkDiscountPricing>(2, 0.10));
    cout << "Bulk discount total: $" << cart.checkout() << endl;

    cout << "\n=== Observer Pattern ===" << endl;
    Subject orderSystem;
    EmailNotifier email;
    InventoryUpdater inventory;
    AnalyticsLogger analytics;

    orderSystem.attach("ORDER_PLACED", &email);
    orderSystem.attach("ORDER_PLACED", &inventory);
    orderSystem.attach(&analytics);  // All events

    orderSystem.notify("ORDER_PLACED", {
        {"orderId", "123"},
        {"email", "customer@test.com"},
        {"product", "Laptop"}
    });

    cout << "\n=== Decorator Pattern ===" << endl;
    // File -> Compression -> Encryption
    auto source = make_unique<EncryptionDecorator>(
        make_unique<CompressionDecorator>(
            make_unique<FileDataSource>("data.txt")
        )
    );

    source->write("Secret data");
    cout << source->read() << endl;
```

```

    return 0;
}
```

Chapter 4

The Project That Ties Everything Together

4.1 Requirements: Build a Configurable Report Generator

Evolution: From Mess to Masterpiece

The Challenge: Aisa system banao jo:

- Different data sources se read kare (CSV, JSON, Database)
- Different formats mein export kare (PDF, Excel, HTML)
- Different transformations apply kare (filter, sort, aggregate)
- Plugin architecture ho (naye formats easily add ho sakein)
- Proper error handling ho (koi source fail ho to gracefully handle)
- Logging aur monitoring ho
- Configurable ho JSON file se

Yeh project aapko seekhayega:

- Real architecture design
- Interface segregation
- Dependency injection
- Error propagation strategies
- Resource management

```
// report_generator.hpp - The Complete System
#ifndef REPORT_GENERATOR_HPP
#define REPORT_GENERATOR_HPP
```

```

#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <memory>
#include <functional>
#include <fstream>
#include <nlohmann/json.hpp>

using json = nlohmann::json;
using namespace std;

// ==================== CORE DATA STRUCTURES ====================

struct DataRow {
    map<string, string> fields;

    string get(const string& col) const {
        auto it = fields.find(col);
        return it != fields.end() ? it->second : "";
    }

    void set(const string& col, const string& val) {
        fields[col] = val;
    }
};

struct DataSet {
    vector<string> columns;
    vector<DataRow> rows;

    bool empty() const { return rows.empty(); }
    size_t size() const { return rows.size(); }
};

// ==================== RESULT TYPE ====================

template<typename T>
class OperationResult {
    bool success;
    T value;
    string error;
    vector<string> warnings;

public:
    OperationResult(T val) : success(true), value(val) {}
    OperationResult(const string& err, const vector<string>& warns
       = {})
        : success(false), error(err), warnings(warns) {}

```

```
    bool isSuccess() const { return success; }
    T getValue() const { return value; }
    string getError() const { return error; }
    vector<string> getWarnings() const { return warnings; }

    static OperationResult<T> ok(T val) { return OperationResult(val); }
    static OperationResult<T> fail(const string& err) { return OperationResult(err); }
};

// ==================== INTERFACES (The Contracts) ====================

class DataSource {
public:
    virtual ~DataSource() = default;
    virtual OperationResult<DataSet> read(const map<string, string>& config) = 0;
    virtual string getName() const = 0;
    virtual bool isAvailable() const = 0;
};

class DataTransformer {
public:
    virtual ~DataTransformer() = default;
    virtual OperationResult<DataSet> transform(const DataSet& input,
                                               const map<string, string>& config) = 0;
    virtual string getName() const = 0;
};

class DataExporter {
public:
    virtual ~DataExporter() = default;
    virtual OperationResult<string> exportData(const DataSet& data,
                                               const map<string, string>& config) = 0;
    virtual string getName() const = 0;
    virtual vector<string> getSupportedFormats() const = 0;
};

// ==================== LOGGER (Observer Pattern) ====================

class LogObserver {
public:
    virtual void onLog(const string& level, const string& message,
```

```
                        const map<string, string>& context) = 0;
    virtual ~LogObserver() = default;
};

class Logger {
private:
    vector<LogObserver*> observers;
    string minLevel;
    map<string, int> levelPriority = {
        {"DEBUG", 0}, {"INFO", 1}, {"WARN", 2}, {"ERROR", 3}, {"FATAL", 4}
    };

public:
    explicit Logger(const string& level = "INFO") : minLevel(level) {}

    void attach(LogObserver* obs) { observers.push_back(obs); }

    void log(const string& level, const string& message,
             const map<string, string>& context = {}) {
        if(levelPriority[level] < levelPriority[minLevel]) return;

        for(auto* obs : observers) {
            obs->onLog(level, message, context);
        }
    }

    void debug(const string& msg, const map<string, string>& ctx = {}) {
        log("DEBUG", msg, ctx);
    }
    void info(const string& msg, const map<string, string>& ctx = {}) {
        log("INFO", msg, ctx);
    }
    void warn(const string& msg, const map<string, string>& ctx = {}) {
        log("WARN", msg, ctx);
    }
    void error(const string& msg, const map<string, string>& ctx = {}) {
        log("ERROR", msg, ctx);
    }
};

// ==================== CONCRETE IMPLEMENTATIONS ====================

// CSV Data Source
class CSVDataSource : public DataSource {
```

```
    Logger* logger;

public:
    explicit CSVDataSource(Logger* log) : logger(log) {}

    string getName() const override { return "CSV"; }

    bool isAvailable() const override {
        return true;  // Always available if file exists
    }

    OperationResult<DataSet> read(const map<string, string>& config
        ) override {
        SCOPE_LOG("CSVDataSource::read");

        auto it = config.find("filepath");
        if(it == config.end()) {
            return OperationResult<DataSet>::fail("CSV: filepath
                not specified");
        }

        string filepath = it->second;
        logger->info("Reading CSV: " + filepath);

        ifstream file(filepath);
        if(!file.is_open()) {
            return OperationResult<DataSet>::fail("CSV: Cannot open
                file " + filepath);
        }

        DataSet data;
        string line;

        // Read header
        if(getline(file, line)) {
            stringstream ss(line);
            string col;
            while(getline(ss, col, ',')) {
                data.columns.push_back(col);
            }
        }

        // Read data
        while(getline(file, line)) {
            DataRow row;
            stringstream ss(line);
            string cell;
            size_t colIdx = 0;

            while(getline(ss, cell, ',') && colIdx < data.columns.
                size()) {
```

```
                row.set(data.columns[colIdx], cell);
                colIdx++;
            }

            data.rows.push_back(row);
        }

        logger->info("CSV read complete: " + to_string(data.rows.
           size()) + " rows");
        return OperationResult<DataSet>::ok(data);
    }
};

// JSON Data Source
class JSONDataSource : public DataSource {
    Logger* logger;

public:
    explicit JSONDataSource(Logger* log) : logger(log) {}

    string getName() const override { return "JSON"; }
    bool isAvailable() const override { return true; }

    OperationResult<DataSet> read(const map<string, string>& config
       ) override {
        string filepath = config.at("filepath");
        logger->info("Reading JSON: " + filepath);

        ifstream file(filepath);
        if(!file.is_open()) {
            return OperationResult<DataSet>::fail("JSON: File not
               found");
        }

        try {
            json j;
            file >> j;

            DataSet data;
            if(j.is_array() && !j.empty()) {
                // Infer columns from first object
                for(auto& [key, val] : j[0].items()) {
                    data.columns.push_back(key);
                }

                for(const auto& item : j) {
                    DataRow row;
                    for(const auto& col : data.columns) {
                        row.set(col, item.value(col, ""));
                    }
                    data.rows.push_back(row);
```

```
                }
            }

            return OperationResult<DataSet>::ok(data);
        } catch(const exception& e) {
            return OperationResult<DataSet>::fail(string("JSON parse error: ") + e.what());
        }
    }
};

// Filter Transformer
class FilterTransformer : public DataTransformer {
    Logger* logger;

public:
    explicit FilterTransformer(Logger* log) : logger(log) {}

    string getName() const override { return "Filter"; }

    OperationResult<DataSet> transform(const DataSet& input,
                                       const map<string, string>& config) override {
        logger->info("Applying filter");

        string column = config.at("column");
        string op = config.at("operator");
        string value = config.at("value");

        DataSet output;
        output.columns = input.columns;

        for(const auto& row : input.rows) {
            string cell = row.get(column);
            bool include = false;

            if(op == "eq") include = (cell == value);
            else if(op == "ne") include = (cell != value);
            else if(op == "contains") include = (cell.find(value) != string::npos);
            else if(op == "gt") include = (stod(cell) > stod(value));
            else if(op == "lt") include = (stod(cell) < stod(value));

            if(include) output.rows.push_back(row);
        }

        logger->info("Filter: " + to_string(input.rows.size()) + " -> " +
                     to_string(output.rows.size()));
```

```
        return OperationResult<DataSet>::ok(output);
    }
};

// HTML Exporter
class HTMLExporter : public DataExporter {
    Logger* logger;

public:
    explicit HTMLExporter(Logger* log) : logger(log) {}

    string getName() const override { return "HTML"; }
    vector<string> getSupportedFormats() const override { return {"
       html", "htm"}; }

    OperationResult<string> exportData(const DataSet& data,
                                       const map<string, string>&
                                          config) override {
        logger->info("Exporting to HTML");

        ostringstream html;
        html << "<!DOCTYPE html><html><head>";
        html << "<style>";
        html << "table{border-collapse:collapse;width:100%}";
        html << "th,td{border:1px solid #ddd;padding:8px;text-align
           :left}";
        html << "th{background-color:#4CAF50;color:white}";
        html << "tr:nth-child(even){background-color:#f2f2f2}";
        html << "</style></head><body>";
        html << "<h2>Report</h2><table><tr>";

        for(const auto& col : data.columns) {
            html << "<th>" << col << "</th>";
        }
        html << "</tr>";

        for(const auto& row : data.rows) {
            html << "<tr>";
            for(const auto& col : data.columns) {
                html << "<td>" << row.get(col) << "</td>";
            }
            html << "</tr>";
        }

        html << "</table></body></html>";

        // Write to file
        string outputPath = config.at("outputPath");
        ofstream out(outputPath);
        out << html.str();

```

```
        return OperationResult<string>::ok(outputPath);
    }
};

// ==================== THE ORCHESTRATOR ====================

class ReportEngine {
private:
    map<string, unique_ptr<DataSource>> sources;
    map<string, unique_ptr<DataTransformer>> transformers;
    map<string, unique_ptr<DataExporter>> exporters;
    Logger logger;

public:
    ReportEngine() {
        // Setup console logging
        struct ConsoleLogger : public LogObserver {
            void onLog(const string& level, const string& msg,
                       const map<string, string>& ctx) override {
                cout << "[" << level << "] " << msg << endl;
            }
        };
        logger.attach(new ConsoleLogger());

        // Register built-in plugins
        registerSource("csv", make_unique<CSVDataSource>(&logger));
        registerSource("json", make_unique<JSONDataSource>(&logger));
        registerTransformer("filter", make_unique<FilterTransformer>(&logger));
        registerExporter("html", make_unique<HTMLExporter>(&logger));
    }

    void registerSource(const string& name, unique_ptr<DataSource> source) {
        sources[name] = move(source);
    }

    void registerTransformer(const string& name, unique_ptr<DataTransformer> trans) {
        transformers[name] = move(trans);
    }

    void registerExporter(const string& name, unique_ptr<DataExporter> exp) {
        exporters[name] = move(exp);
    }

    OperationResult<string> generateReport(const json& config) {
        logger.info("Starting report generation");
```

```
372
373         // Parse config
374         string sourceType = config["source"]["type"];
375         string exporterType = config["exporter"]["type"];
376
377         // Validate
378         if(sources.find(sourceType) == sources.end()) {
379             return OperationResult<string>::fail("Unknown source: "
                    + sourceType);
380         }
381         if(exporters.find(exporterType) == exporters.end()) {
382             return OperationResult<string>::fail("Unknown exporter:
                    " + exporterType);
383         }
384
385         // Execute pipeline
386         // 1. Read
387         map<string, string> sourceConfig = config["source"]["config
                "];
388         auto readResult = sources[sourceType]->read(sourceConfig);
389         if(readResult.isFailure()) {
390             return OperationResult<string>::fail("Read failed: " +
                    readResult.getError());
391         }
392
393         DataSet data = readResult.getValue();
394
395         // 2. Transform
396         if(config.contains("transformations")) {
397             for(const auto& trans : config["transformations"]) {
398                 string transType = trans["type"];
399                 if(transformers.find(transType) == transformers.end
                        ()) {
400                     logger.warn("Unknown transformer: " + transType
                            );
401                     continue;
402                 }
403
404                 map<string, string> transConfig = trans["config"];
405                 auto transResult = transformers[transType]->
                        transform(data, transConfig);
406                 if(transResult.isFailure()) {
407                     logger.warn("Transform failed: " + transResult.
                            getError());
408                     continue;
409                 }
410                 data = transResult.getValue();
411             }
412         }
413
414         // 3. Export
```

```
        map<string, string> exportConfig = config["exporter"]["
            config"];
        auto exportResult = exporters[exporterType]->exportData(
            data, exportConfig);
        if(exportResult.isFailure()) {
            return OperationResult<string>::fail("Export failed: "
                + exportResult.getError());
        }

        logger.info("Report generation complete: " + exportResult.
            getValue());
        return OperationResult<string>::ok(exportResult.getValue())
            ;
    }
};

#endif
```

```
// main_report.cpp
#include "report_generator.hpp"

int main() {
    ReportEngine engine;

    // Create sample data files
    {
        ofstream csv("sales.csv");
        csv << "Product,Region,Amount,Date\n";
        csv << "Laptop,North,1200,2024-01-15\n";
        csv << "Mouse,North,25,2024-01-16\n";
        csv << "Laptop,South,1200,2024-01-17\n";
        csv << "Keyboard,East,100,2024-01-18\n";
        csv << "Monitor,North,300,2024-01-19\n";
    }

    // Configuration
    json config = {
        {"source", {
            {"type", "csv"},
            {"config", {{"filepath", "sales.csv"}}}
        }},
        {"transformations", {
            {
                {"type", "filter"},
                {"config", {
                    {"column", "Region"},
                    {"operator", "eq"},
                    {"value", "North"}
                }}
            }
        }},
```

```
        {"exporter", {
            {"type", "html"},
            {"config", {{"outputPath", "report.html"}}}
        }}
    };

    cout << "=== Report Generator Demo ===" << endl;

    auto result = engine.generateReport(config);

    if(result.isSuccess()) {
        cout << "\n   Report generated: " << result.getValue() << endl;
    } else {
        cout << "\n   Failed: " << result.getError() << endl;
    }

    return 0;
}
```

Phase 2 Complete!

Aapne seekha:

1. **Defensive Programming:** Result pattern, transactional safety
2. **Debugging:** Tools, mindset, scientific method
3. **Design Patterns:** Real usage, not theory
4. **Architecture:** Plugin system, dependency injection
5. **Error Handling:** Graceful degradation, logging

Ab aap confidently keh sakte ho:

- "Yeh code production-ready hai"
- "Main jaanta hoon yeh kahan fail sakta hai"
- "Mere paas debugging strategy hai"

Think. Design. Then Code.

"The code you write today will be debugged by someone at 3 AM. Be kind."